

Foundation — Sub-project 1 Design

Foundation — Sub-project 1 Design

Field	Value
Revision	1
Created	2026-05-20
Last modified	2026-05-20
Status	active
Status summary	Approved design for Foundation (Sub-project 1 of 8) composing HRD-018 (commons_constitution) + HRD-010 (Postgres + River) + HRD-016 (Gin RESt skeleton) + HRD-026 (bundle-hash captureer) + HRD-027 (mode-ladder runtime config) + HRD-028 (/v1/compliance pull surface). Approach B (bottom-up vertical slices) across three milestones M1/M2/M3. Each milestone closes its own end-to-end smoke test.
Issues	HRD-018, HRD-010, HRD-016, HRD-026, HRD-027, HRD-028
Issues summary	see ../../Issues.md
Fixed	none yet (Foundation is in_progress)
Fixed summary	—
Continuation	M1 commons_constitution scaffold next. Catalogue-Check survey gate per Universal §11.4.74 MUST complete before any Go code lands.

Table of contents

- [§0. Scope](#)
- [§1. Architecture overview](#)
- [§2. Components](#)
- [§3. Data flow](#)
- [§4. Error handling](#)
- [§5. Testing](#)
- [§6. Out of scope](#)
- [§7. Open questions](#)
- [§8. Spec landing — §44 of specification.V3.md](#)
- [§9. Addendum — Catalogue-Check pivot \(2026-05-20\)](#)

§0. Scope

Foundation is **Sub-project 1 of 8** in the Herald V3 implementation roadmap. It delivers the substrate every other sub-project will build on: the constitution-rule evaluator framework, the governance event taxonomy, the persistent state + audit trail, the per-tenant mode-ladder runtime config, and the minimum REST surface required to push events in and pull compliance state out.

Composes these HRDs:

- **HRD-018** — commons_constitution package scaffold.
- **HRD-010** — commons_storage live wiring (pgx + golang-migrate + River + Redis).
- **HRD-016** — Gin REST skeleton (subset of §41).
- **HRD-026** — Bundle-hash captureer.
- **HRD-027** — Mode-ladder runtime config (Postgres source-of-truth + Redis cache).
- **HRD-028** — /v1/compliance pull surface.

Done criterion (locked): The Quickstart compose stack accepts a real CloudEvent on POST /v1/events, fans it out to the null:// channel, writes a constitution_state row with the correct transition, and exposes that row via GET /v1/compliance.

Evaluator trigger model (locked): Hybrid — push for critical-severity rules (§9 secret-handling, §11.4.10 root-cause, §12 ops invariants), pull-sweep every 5–15 min for lower-severity rules (sweep daemon itself lives in Sub-project 5, but the building blocks land here).

Mode-ladder storage (locked): Postgres constitution_bindings table as source-of-truth + Redis 60 s read-cache (mirrors §4.3 idempotency-keys pattern).

Approach (locked): B — bottom-up vertical slices across three milestones, each its own PR-sized vertical slice that closes its smoke test before the next milestone starts.

§1. Architecture overview

Three milestones, one-way module dependency direction (per spec §10 layering):

pherald —> commons_constitution —> commons_messaging +
commons_storage —> commons

commons_constitution lives at L1 as a sibling of commons_messaging and commons_storage — no cycle.

M1 – Pure-Go core (no infra, no network)

commons_constitution/	← NEW package
├─ evaluator.go	Evaluator interface + Registry
├─ emit.go	12 event-class emit helpers (per §42.2)
├─ bundle.go	BundleHash captureer (§42.1.3)
├─ ladder.go	ModeLadder interface + in-memory

```
impl
  └─ state/memory.go          in-memory ConstitutionStore (test
backend)
  commons_messaging/
    └─ router_memory.go      ← extends r1
                             Watermill memory-pubsub wire-up
```

M1 smoke = `go test ./commons\_constitution/...` proves an in-memory evaluator detects a transition → emits a .policy-violation event → memory-pubsub listener counts it.

M2 – Postgres + River live

```
commons_storage/
  └─ pgx.go                  ← extends r1
                             pgx pool + RLS tenant-context
middleware
  └─ migrate.go              golang-migrate driver wired to
embed FS
  └─ river.go                River queue setup + worker
registration
  └─ migrations/000006_constitution_state.up.sql      NEW
  └─ migrations/000007_constitution_bindings.up.sql  NEW
commons_constitution/state/postgres.go ← replaces memory
backend
commons_constitution/ladder/postgres.go ← Postgres ladder
```

M2 smoke = `go test -tags=integration ./...` against testcontainers
 Postgres + River verifies: constitution_state row written; transition detected via SHA compare; River job enqueued + consumed; audit row appended.

M3 – Gin REST + Redis cache

```
pherald/internal/http/
  └─ server.go              ← NEW directory
                             Gin engine + graceful shutdown
($3.1)
  └─ ingest.go              POST /v1/events (CloudEvents
binary + structured)
  └─ compliance.go          GET /v1/compliance (pull surface
$42.1.5)
  └─ middleware.go          JWT + RLS tenant-context + OTel
commons_storage/redis.go    ← Redis client +
ACL
commons_constitution/ladder/redis_cache.go ← 60 s read-cache
wrapper
```

M3 smoke = `podman-compose up`; curl POST /v1/events → fans to null://
 → constitution_state row written → diary appended → GET /v1/compliance returns the row. Matches the locked done-criterion.

Per Universal §11.4.74 (catalogue-check), each milestone starts with a survey of vasic-digital + HelixDevelopment orgs. Expected verdicts:

- **M1:** no-match — constitution-rule evaluation semantics are bespoke to Helix.
- **M2:** extend — golang-migrate + River + pgx are external standards we wrap, not re-implement.
- **M3:** extend — Gin is the external standard.

The actual verdict for HRD-018 lands in docs/Issues.md before any Go code is written.

§2. Components

§2.1 commons_constitution package (L1)

The core abstraction layer for constitution-rule evaluation + event emission.

```
// evaluator.go
type Severity int
const (
    SeverityLow Severity = iota
    SeverityMiddle
    SeverityHigh
    SeverityCritical
)

type Decision int
const (
    DecisionPass Decision = iota
    DecisionWarn
    DecisionFail
    DecisionError
    DecisionSkip
)

type Subject struct {
    Kind string // "file" / "repo" / "commit" / "tenant" /
                "release-tag"
    ID    string // path, repo URL, SHA, tenant UUID, ref
}

type Result struct {
    Decision Decision
    Evidence string // URI to captured-evidence artefact
                (§11.4.2)
    DigestSHA [32]byte // hash of evaluator's output; transition
                detected on change
}

type Evaluator interface {
    RuleID() string // "§11.4.10"
    Severity() Severity
```

```

    PushTriggers() []string // CloudEvents
                             types that cause re-eval
    Subjects(ctx context.Context, tenantID uuid.UUID) ([]Subject,
        error)
    Evaluate(ctx context.Context, s Subject, bundle BundleHash)
        (Result, error)
}

type Registry struct
    { /* sync.RWMutex-guarded map keyed by RuleID */ }
func (r *Registry) Register(e Evaluator)
func (r *Registry) Get(ruleID string) (Evaluator, bool)
func (r *Registry) IterByMaxSeverity(max Severity)
    iter.Seq[Evaluator]

// emit.go – typed wrappers around 12 event classes (§42.2)
type EventEmitter interface {
    GateFailed(ctx context.Context, e GateEvent) error
    GateRecovered(ctx context.Context, e GateEvent) error
    PolicyViolation(ctx context.Context, e PolicyEvent) error
    PolicyCleared(ctx context.Context, e PolicyEvent) error
    HostSafetyBreach(ctx context.Context, e SafetyEvent) error
    RepoSafetyBreach(ctx context.Context, e SafetyEvent) error
    CredentialLeak(ctx context.Context, e CredentialEvent) error
    BundleUpdated(ctx context.Context, e BundleEvent) error
    BundleUpdateFailed(ctx context.Context, e BundleEvent) error
    ReleaseGateBlocked(ctx context.Context, e ReleaseEvent) error
    SpecRevisionDrift(ctx context.Context, e DriftEvent) error
    CatalogueMiss(ctx context.Context, e CatalogueEvent) error
}

// All emitted CloudEvents carry the §42.1.1 three-axis envelope:
// (rule_id, severity_category, decision_result)
// plus bundle_hash + transition_from→to + traceparent + evidence
// URI.

// bundle.go – §42.1.3
type BundleHash [32]byte
func Capture(constitutionMdPath string) (BundleHash, error) //
    SHA-256 of rendered MD
func (h BundleHash) String() string //
    hex-encoded

// ladder.go – §42.1.4
type Mode int
const (
    ModeAllow Mode = iota // record state; no audit; no channel
        emit
    ModeWarn // record state; audit; no channel
        emit
    ModeEnforce // record state; audit; channel emit
)

type ModeLadder interface {
    Get(ctx context.Context, tenantID uuid.UUID, ruleID string)
        (Mode, error)
}

```

```

    Set(ctx context.Context, tenantID uuid.UUID, ruleID string, m
        Mode, by string) error
    List(ctx context.Context, tenantID uuid.UUID)
        (map[string]Mode, error)
}

// state/state.go – §42.1.2
type Transition struct {
    OldDecision Decision
    NewDecision Decision
    OldDigest    [32]byte
    NewDigest    [32]byte
    Changed      bool    // true iff (decision changed) OR (digest
        changed) OR (bundle_hash changed)
}

type ConstitutionStore interface {
    Record(
        ctx context.Context,
        tenantID uuid.UUID,
        ruleID, subject string,
        r Result,
        bundle BundleHash,
    ) (Transition, error)
}

```

Backend implementations:

Backend	M1	M2	M3	Notes
state/memory.go	✓	(test-only)	(test-only)	sync.Map keyed by (tenant, rule, subject).
state/postgres.go	—	✓	✓	RLS-guarded UPSERT on constitution_state.
ladder/memory.go	✓	(test-only)	(test-only)	for unit tests.
ladder/postgres.go	—	✓	✓	source-of-truth.
ladder/redis_cache.go	—	—	✓	wraps ladder/postgres.go; 60 s TTL; cache-aside; invalidate on Set.

§2.2 commons_storage additions (M2)

Builds on the r1 stub. New files:

- **pgx.go** — pgx pool config; WithTenantContext(ctx, tenantID) wrapper that runs BEGIN; SET LOCAL app.tenant_id = '<uuid>'; per transaction (§16).
- **migrate.go** — wires github.com/golang-migrate/migrate/v4 source + database drivers to consume the embedded migrations/ FS shipped in r1; implements the MigrationDriver interface stub.

- **river.go** — River queue config; one default worker pool; helper to register workers from a flavor.
- **redis.go** — go-redis client; per-tenant key prefix `t:<id>:`; ACL-user auth from `[redis.acl]` config block.

New migrations:

- **000006_constitution_state.up.sql** (+ `.down.sql`):

```
CREATE TABLE constitution_state (
    tenant_id      UUID NOT NULL REFERENCES tenants(id),
    rule_id        TEXT NOT NULL,
    subject        TEXT NOT NULL,
    decision       SMALLINT NOT NULL,
    digest_sha     BYTEA NOT NULL,
    bundle_hash    BYTEA NOT NULL,
    evidence_uri   TEXT,
    transitioned_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (tenant_id, rule_id, subject)
);
CREATE INDEX constitution_state_decision_idx
    ON constitution_state (tenant_id, decision,
        transitioned_at DESC);
ALTER TABLE constitution_state ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON constitution_state
    USING (tenant_id = current_setting('app.tenant_id')::uuid);

CREATE TABLE constitution_audit (
    id             UUID PRIMARY KEY DEFAULT uuidv7(),
    tenant_id      UUID NOT NULL REFERENCES tenants(id),
    rule_id        TEXT NOT NULL,
    subject        TEXT NOT NULL,
    old_decision   SMALLINT,
    new_decision   SMALLINT NOT NULL,
    bundle_hash    BYTEA NOT NULL,
    evidence_uri   TEXT,
    emitted_event_id UUID,
    audited_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX constitution_audit_lookup_idx
    ON constitution_audit (tenant_id, rule_id, audited_at
        DESC);
ALTER TABLE constitution_audit ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON constitution_audit
    USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

- **000007_constitution_bindings.up.sql** (+ `.down.sql`):

```
CREATE TABLE constitution_bindings (
    tenant_id      UUID NOT NULL REFERENCES tenants(id),
    rule_id        TEXT NOT NULL,
    mode           SMALLINT NOT NULL,    -- 0 allow / 1 warn / 2
        enforce
    mutated_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```

    mutated_by TEXT NOT NULL,
    PRIMARY KEY (tenant_id, rule_id)
);
ALTER TABLE constitution_bindings ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON constitution_bindings
    USING (tenant_id = current_setting('app.tenant_id')::uuid);

```

§2.3 pherald/internal/http Gin REST layer (M3)

Net-new directory under pherald. Files:

- **server.go** — Gin engine wired on `[server].http_port` (default 24091 per §9.4); graceful-shutdown handler per §3.1 (traps SIGTERM, drains in-flight, exits 0/1).
- **ingest.go** — `POST /v1/events` accepts CloudEvents in both modes (binary headers + structured JSON body) via `github.com/cloudevents/sdk-go/v2`.
- **compliance.go** — `GET /v1/compliance` returns `constitution_state` rows; query params `rule`, `subject`, `decision`; cursor pagination.
- **middleware.go** — three middlewares: JWT-bearer auth (validates against JWKS in `[auth.oidc]`), RLS-tenant-context (extracts `tenant_id` claim → sets `pgx` transaction context), OTel HTTP (span per request + `http.server.duration` histogram).

Routes mounted at M3 (intentionally minimal — full §41 contract lands in HRD-016 expansion in Sub-projects 2–5):

Method	Path	Purpose	Milestone
POST	<code>/v1/events</code>	CloudEvents ingest	M3
GET	<code>/v1/compliance</code>	Constitution-state pull surface	M3
GET	<code>/v1/healthz</code>	Composes with §17.5 probes	M3
GET	<code>/metrics</code>	Prometheus scrape	M3

§3. Data flow

§3.1 Push trigger — critical severity (M3 happy path)

Trace of a single rule-violation, end-to-end:

[1] gherald detects ``git tag v1.4.0`` attempted without SLISA L3 attestation.

[2] gherald composes a CloudEvent `v1.0`:

```

ce-id:          01HXM...UUIDv7
ce-source:      digital.vasic.herald/gherald
ce-type:        digital.vasic.herald.git.release.tag-attempted
ce-subject:     repo:vasic-digital/Herald#v1.4.0
ce-time:        2026-05-20T15:42:11.123Z
ce-extensions:  tenantid=<uuid>, bundlehash=<sha256>,
traceparent=...
data:           { "ref": "v1.4.0", "sha": "abc...",

```



```
"attestations": [] }
```

[3] POST /v1/events → Gin handler ingest.go
– cloudevents.NewHTTPReceiver decodes (binary or structured mode)

– JWT middleware validates bearer → claim.tenantid
– cross-checks ce.extensions.tenantid == claim.tenantid (defence-in-depth)
– middleware.go calls pgx.WithTenantContext(ctx, claim.tenantid)
→ BEGIN; SET LOCAL app.tenant_id = '<uuid>';

[4] idempotency_keys UPSERT on ce-id → if duplicate, return 200 immediately.

[5] commons_messaging.Router.Publish(ctx, ce) →
Watermill in-process pubsub → subscriber
"constitution_evaluator" wakes.

[6] Evaluator path (commons_constitution):
a. Look up bindings: §42 catalogue maps "release.tag-attempted" →
evaluators [§11.4.65 release-gate, §11.4.10 supply-chain].
b. For each evaluator E:
mode := ModelLadder.Get(ctx, tenantID, E.RuleID())
↑ Redis hit (60s TTL) | miss → Postgres
constitution_bindings
bundle := bundle.Capture(constitutionPath) (in-mem
cached per process)
result := E.Evaluate(ctx, subject, bundle)
↑ returns Result{Decision: Fail, Evidence: "no SLSA
attestation"}
trans := ConstitutionStore.Record(ctx, tenantID,
E.RuleID(),
ce.subject, result, bundle)
↑ UPSERT constitution_state PK(tenant, rule,
subject);
returns Transition{OldDecision: Pass, NewDecision:
Fail, Changed: true}
// NOTE: state is ALWAYS recorded regardless of mode. Mode
only gates
// what happens AFTER the transition is detected
(step [7]).

[7] Transition gate – THIS IS THE CORE OF THE DESIGN:
if !trans.Changed: return // §42.2 "transitions-
only" discipline
switch mode {
case ModeAllow: return //
recorded; no audit, no emit
case ModeWarn: auditOnly(ctx, trans, E) // audit
only, no channel emit
case ModeEnforce: emit(ctx, trans, E) // audit +
channel emit

```
}
```

- [8] emit() path:
- Compose outbound CloudEvent of type `.release-gate-blocked` carrying:
 - three-axis envelope: (rule_id, severity_category, decision_result)
 - bundle_hash, transition_from→to, evidence URI, traceparent
 - Append audit row to `constitution_audit` (insert-only).
 - `River.Insert(channelFanoutJob{event, subscribers})` → worker picks up, looks up `channel_addresses` + `preference_sets` per subscriber, dispatches via `commons_messaging.Send()`.
 - M3 dev: only `null://` is wired → ring-buffer captures the message.
- [9] Reply path:
- 202 Accepted to gherald with `ce-id`.
 - response body: `{"event_id": "01HXM...", "decision": "fail", "mode": "enforce", "transition": "pass→fail"}`.

§3.2 Pull trigger — non-critical severity (scherald sweep, deferred)

Foundation lays the foundation but does NOT ship the sweep daemon — that's Sub-project 5. However the M3 surface exposes the trigger path:

```
scherald (cron, every 5–15 min)
  → for each evaluator E in Registry where E.Severity() < SeverityCritical:
    → POST /v1/compliance/sweep (admin route, mounted in Sub-project 5)
    → handler iterates E.Subjects() and runs the same [6]...[8] flow.
```

For Foundation we expose the **building blocks** (`Registry.IterByMaxSeverity`, `Evaluator.Subjects`, the same `Evaluator` + `Record` + `Transition gate`) — and verify them with a unit test that simulates the sweep loop in-process.

§3.3 Compliance pull surface

```
GET /v1/compliance?rule=$1.4.10&decision=fail&cursor=...&limit=50
  → JWT auth → tenant claim → RLS context
  → SELECT rule_id, subject, decision, transitioned_at,
evidence_uri,
      bundle_hash, mode
FROM constitution_state
WHERE rule_id = $1 AND decision = $2
ORDER BY transitioned_at DESC
LIMIT $3
  → JSON page envelope {data: [...], next_cursor: ...}
```

RLS guarantees only the caller's tenant rows are visible — no application-level WHERE tenant_id = clause needed (and a missing one would still return zero rows, not a leak).

§3.4 Bundle-hash invalidation cascade

When <ancestor>/constitution/Constitution.md is mutated:

1. bundle.Capture() returns a new SHA-256.
2. Process-local cache invalidated on next call (TTL or explicit poke).
3. All next-call Results carry the new bundle hash.
4. ConstitutionStore.Record compares (old_bundle_hash, new_bundle_hash) — if changed and Decision is unchanged, **a transition is still emitted** (.bundle-updated event) because the rationale changed even if the verdict didn't.

This is the §42.2 mandate that says "evaluator stability across bundle revisions is itself a tracked event."

§4. Error handling

Failure mode	Behavior	Rationale
Evaluator panics	recover() in Registry.runOne; Record Decision: Error; audit; no channel emit	Don't spam channels with our own bugs (§40.3 challenge panic-isolation).
Postgres unavailable	River + pgx pool retries with exponential backoff; if persistent, /v1/events returns 503	Caller retries via own idempotency key. Pull surface degrades to 503; push events buffer in River up to its capacity.
Redis unavailable	ladder/redis_cache.go falls through to ladder/postgres.go; log cache_miss_fallback metric	Cache is a perf optimisation, never load-bearing for correctness.
Postgres AND Redis down	Default to ModeWarn (conservative): audit-only, no channel emit, no allow	Avoid spamming channels in degraded state; don't silently allow either.
Constitution bundle unreadable	Evaluator returns Decision: Skip; emit single .catalogue-miss event per tenant per process lifetime (dedup via in-memory set; reset on process restart)	One alert per outage, not one per evaluation. Missing bundle has no SHA to key on.
Missing tenant claim in JWT	401 at middleware.go; never reaches Evaluator	Defence in depth; RLS is a second wall, not the first.

Failure mode	Behavior	Rationale
Idempotency-key replay	INSERT ... ON CONFLICT DO NOTHING → 200 with prior result	§4.3 idempotency contract.
River job DLQ overflow	Surface via /v1/healthz (degraded) + Prometheus river_job_dlq_total	Operator decides retry/discard.
Bundle file mutated mid-sweep	New SHA captured on next Capture() call; in-flight evaluation completes with old SHA + emits .bundle-updated transition on next round	Stable read for each evaluation; mutation eventually consistent.

Invariant: No code path may both (a) detect a constitution violation and (b) consume the violation in the same function. Detection and emission must be separated by the Registry boundary — this is what makes the panic-recovery in row 1 trustworthy.

§5. Testing

Three tiers, mapped to the milestones:

§5.1 M1 unit tests (table-driven, in-process)

Test	What it proves
Evaluator.Evaluate correctness	given subject + bundle → produces expected Result
BundleHash.Capture determinism	same input → same SHA; one-byte mutation → different SHA
ModeLadder.memory (Get/Set/List)	atomicity under concurrent Get + Set
Transition gate	Changed=true iff Decision changed OR DigestSHA changed OR BundleHash changed
Registry.runOne panic isolation	evaluator that always panics → Decision: Error, no other evaluators affected
Memory-pubsub emit round-trip	publish 1000 events → listener consumes 1000 events in order

§5.2 M2 integration tests (//go:build integration, testcontainers)

Test	What it proves
ConstitutionStore.Record UPSERT semantics	INSERT-then-UPDATE returns Transition{Changed: true}; UPSERT same value returns Transition{Changed: false}
RLS tenant isolation	

Test	What it proves
	Tenant A's INSERT not visible to Tenant B's SELECT even without WHERE clause
Migrations 000006/000007 up/down	Idempotent — run up + down + up; verify table state
golang-migrate driver wired through embed FS	MigrationDriver.Up() runs the 7 migrations end-to-end
River enqueue → consume → audit	Enqueue 100 jobs, all consumed, 100 audit rows appended
pgx WithTenantContext	Subsequent current_setting('app.tenant_id') returns the set UUID

§5.3 M3 end-to-end smoke (containerised)

Manual + scripted via containers/quickstart/:

```
podman-compose -f docker-compose.quickstart.yml up -d
curl --retry 30 --retry-delay 2 http://localhost:24091/v1/healthz
curl -X POST http://localhost:24091/v1/events \
  -H "Authorization: Bearer <dev-jwt>" \
  -H "Content-Type: application/cloudevents+json" \
  -d @testdata/release-tag-attempted.ce.json
curl -H "Authorization: Bearer <dev-jwt>" \
  "http://localhost:24091/v1/compliance?
rule=$11.4.65&decision=fail"
# Assert: constitution_state row exists, null:// ring buffer has 1
message,
#      diary/main.md has new entry.
```

§5.4 Mapping to §40.3 named challenges

Challenge ID	Where verified
idempotency-replay	M2 integration (replay same ce-id → 200, no second row)
transition-correctness	M1 table-driven (Changed boolean cases)
rls-isolation	M2 integration (tenant A vs tenant B SELECT)
mode-ladder-respect	M1 + M3 smoke (allow/warn/enforce path coverage)
bundle-hash-drift	M1 unit (mutate fixture → expect .bundle-updated transition)
panic-isolation	M1 unit (panicking evaluator → no listener cascade)
cache-fallback	M3 smoke (kill Redis container → assert continued correctness)
dlq-overflow	M2 integration (saturate River → assert 503 + health-degraded)

Fixtures live under each package's testdata/. No live network in M1+M2. Live network (Telegram, Slack, real Claude Code) is **strictly deferred** to Sub-projects 2/3.

§6. Out of scope

To bound Foundation cleanly:

- **No evaluator implementations.** Registry is populated by Sub-projects 2 onwards (HRD-036/HRD-037/HRD-038/etc).
- **No real channel adapters.** Telegram, Slack, Email, MS Teams, etc. live in Sub-projects 2/3.
- **No Claude Code dispatcher.** Lives in Sub-project 2 (HRD-012).
- **No pherald serve daemon-mode polling loops.** Only the Gin server runs; per-channel Subscribe() loops land in Sub-project 2.
- **No full §41 v1 surface.** Only the four routes above. /v1/items, /v1/subscribers, /v1/channels, /webhooks/* come later.
- **No scherald sweep daemon.** Building blocks land in Foundation; the daemon itself is Sub-project 5.
- **No multi-region deploy or HA story.** Single-region single-leader Postgres + Redis is the M3 default.

§7. Open questions

The following are flagged as open for resolution before each milestone closes:

1. **Bundle path resolution under parent-discovery.** bundle.Capture needs the path to <ancestor>/constitution/Constitution.md. Should this be (a) resolved once at process startup via the §103 walk and cached in a global, or (b) re-resolved per Capture call with TTL? Recommendation: (a) at startup, with a SIGHUP handler to force re-walk. **Resolve before M1 closes.**
2. **River job payload size cap.** River persists job args as JSONB. With a CloudEvent + attachment metadata + trace context, payloads can exceed 1 MB. Recommendation: store the CloudEvent body in inbound_messages (existing table from r1) and pass only the ce-id to the River job. **Resolve before M2 closes.**
3. **Cache-invalidate fan-out on mode-ladder Set.** When ladder.Set mutates a binding, the 60 s Redis TTL means stale reads for up to 60 s. Acceptable for allow→warn (perms tightening) but risky for enforce→allow (perms loosening). Recommendation: on every Set, publish a mode-ladder.invalidated event on a Watermill topic; every running process invalidates its own local cache. **Resolve before M3 closes.**
4. **JWT issuer + JWKS source.** Foundation needs a dev-mode issuer for the M3 smoke. Options: (a) a static dev-only JWKS file under containers/quickstart/, (b) embed a tiny HS256 issuer in pherald itself for dev, (c) require operator to point at an external IdP from day one. Recommendation: (a) — minimum friction, clearly dev-only. **Resolve before M3 closes.**

§8. Spec landing — §44 of specification.V3.md

Per Universal §11.4.73 (spec-versioning), this design is reflected in docs/specs/mvp/specification.V3.md as a new section **§44 Foundation implementation contract**, with the V3 Revision bumped from 6 → 7.

§44 lifts the locked decisions + the three-milestone shape from this design doc, but the *full* design (Sections §1–§7 above) lives only here under docs/superpowers/specs/. The spec carries the contract; the design doc carries the rationale, the data-flow trace, the failure modes, the open questions, and the per-section discussion that led to the contract.

When Foundation lands, the V3 §44 section is updated with the as-built evidence (smoke-test outputs, audit-row examples, real CloudEvent traces) and the design doc's status moves from active to landed.

§9. Addendum — Catalogue-Check pivot (2026-05-20)

The §1–§5 design above was approved before the §11.4.74 Catalogue-Check survey. The survey result is recorded in docs/catalogue-checks/HRD-018-foundation.md and changes the **library choices** but not the **architecture, smoke-test contracts, or data-flow trace**.

§9.1 Library substitutions

§1–§5 reference	Catalogue verdict	Substitute used in implementation
Watermill memory-pubsub	extend	digital.vasic.eventbus
River Postgres queue	EXTEND-REPLACE	digital.vasic.background
Raw Gin server skeleton	strong-extend	digital.vasic.middleware (chain + recovery + requestid + ratelimit + Gin adapter)
Raw JWT + Bearer middleware	strong-extend	digital.vasic.auth (pkg/jwt + pkg/middleware)
Raw pgx pool + golang-migrate wrappers	extend	digital.vasic.database (pkg/postgres + pkg/migration)
Raw go-redis client	extend	digital.vasic.cache
Custom OTel wiring	extend	digital.vasic.observability
Raw config loader	extend	digital.vasic.config
	extend	

§1–§5 reference	Catalogue verdict	Substitute used in implementation
Custom panic-recovery wrapper		digital.vasic.recovery (also re-exported by middleware)

§9.2 What this means for §1–§5

- §1 architecture diagram still holds; module-dependency direction is unchanged.
- §2.2 `commons_storage` `pgx.go` and `migrate.go` collapse into thin adapters around `digital.vasic.database's` `pkg/postgres` and `pkg/migration`. The file names stay (per the constitutional file-naming convention) but their bodies shrink.
- §2.2 `commons_storage` `river.go` is **renamed** `background.go` and adapts `digital.vasic.background.WorkerPool` instead of `River`.
- §2.3 `pherald/internal/http/middleware.go` becomes a 30-line composition of three Helix-stack middleware chains rather than hand-rolled middleware.
- §3.1 step [5] in the data-flow trace reads `"digital.vasic.eventbus.Bus.Publish(ctx, ev)"` rather than `"Watermill in-process pubsub"` — semantics identical.
- §3.1 step [8c] reads `"digital.vasic.background.queue.Enqueue(channelFanoutJob{...})"` rather than `"River.Insert(...)"` — semantics identical.
- §4 error-handling table is unchanged. Panic isolation uses `digital.vasic.recovery` in place of an ad-hoc `recover()`.
- §5 testing tiers are unchanged. Integration tests run against the **same** Postgres testcontainers + the **same** queue semantics — only the implementation library changes.

§9.3 Submodule-installation policy

Per the Herald CLAUDE.md vendored-SDK policy, all 9 modules above are installed as **git submodules under submodules/<lowercase-name>/** rather than go get'd into `go.mod`. The local `go.work` lists each submodule's path; `go.mod` replace directives during development point at those paths. Production builds resolve through pinned commit SHAs.

§9.4 Safety contract (continuous user mandate 2026-05-20)

Every module integration in this Foundation cycle **MUST**:

1. **Fetch + rebase** before any submodule add/update.
2. **Compile + test** before commit — `go build ./...` AND `go test ./...` AND `bash tests/test_constitution_inheritance.sh` (12 PASS / 0 FAIL gate) all green.
3. **Never force-push**; never use `--no-verify` or `--no-gpg-sign`.
4. **Anti-bluff**: every test added in M1/M2/M3 **MUST** actually exercise the behavior it claims to verify — no pass-without-execution paths, no mock-only tests that don't round-trip, no skip-by-default integration tests. Each test failure in CI **MUST** imply a real broken feature.
5. **Hardlinked backup** before any destructive operation (file delete, schema-changing migration, etc).

6. **Multi-mirror fan-out** push to all 4 remotes after each meaningful milestone commit.